Dynamic sites are more complicated than static ones; they need an interface for the users to interact with, somewhere to store data like uploaded pictures and comments, and logic for knowing how to combine these into content. Django's primary purpose is building dynamic websites. In doing so, it uses a common architectural pattern for user interfaces known as the *model—view—controller* (MVC) method. This method breaks software into three pieces, as shown in the following list and figure 1.4:

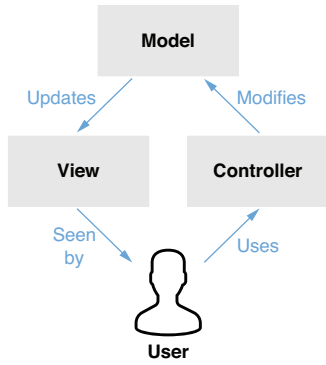


Figure 1.4 Model—view—controller architectural pattern

1. *Model*—Represents the logic and data of an application independent of its user interface
2. *View*—Presents data to the user, possibly with multiple views for the same data
3. *Controller*—An input and control mechanism the user uses to interact with models and views

The MVC is a good way of thinking about how software interacts with users, but Django is more inspired by it than strictly governed by it. The line between what is a view and a controller is a little fuzzy. This fuzziness provides the added benefit of creating entertaining debate on the internet.

When a web server passes a URL request to Django through WSGI, Django looks that URL up in its master mapping file, named urls.py, and as the name implies, it is just Python. The file can be renamed, but there really isn't any reason to do so. The urls.py file contains a variable called `urlpatterns`, which is a `list` containing Django `path` objects. Each `path` object either maps a URL to a Django view or loads another file containing more mappings.

A Django view is just Python code. It can be a function that returns a response document or a class with methods that do the same. Each view takes at least one argument, called the `request`. This variable contains information about the HTTP Request that was made. The request object includes the URL that was hit; any query parameters in the URL; HTTP method information; HTTP headers; and in the case of pages with forms, the data the user filled in. Not every view needs all this information, but it is there when you require it.

Key steps in Django's request handling process straddle the boundaries defined by the MVC. Django mapping a URL to the code that handles it is considered an MVC controller. Django's view, which returns the HTML document, is a hybrid of the MVC view and controller. Inside a Django view, models get queried, which makes it an MVC controller, but it also returns HTML, which makes it an MVC view. Just because Django names it *view* doesn't mean it is an MVC view—don't let the overlapping names confuse you.

Consider the prior song-by-year page example. The same Django view outputs different results, depending on the year passed in. The HTML template used in the